

Experiment 2: Row-Oriented vs. Column-Oriented Query Patterns for the SAW Approval-Gate Audit Trail

Status: completed, measured experiment. This is not design notes — it is a real, executed benchmark with reproducible numbers. It converts the design questions raised in `clickhouse-agent-memory-eval.md` and `ibm-bob-coding-assistant-eval.md` into an actual second experiment for the Sovereign AI Workforce (SAW) project, run alongside the original SEAG experiment.

Critical scope note — read before interpreting any number below

This experiment does not run actual PostgreSQL/pgvector or actual ClickHouse. Neither could be installed in the sandboxed environment used to run this benchmark (no network access to install external database servers). Instead, it tests the underlying *architectural claim* — row-oriented vs. column-oriented access patterns — using available substitutes:

Represents	Actual tool used	Why it's a fair proxy
Row-oriented store (PostgreSQL-style)	SQLite (Python standard library), tested both unindexed and indexed	Same B-tree row-lookup query execution model as PostgreSQL for this query shape
Column-oriented store (ClickHouse-style)	Pandas/NumPy in-memory columnar arrays	Same columnar-scan-and-filter execution model ClickHouse uses for this query shape

Any number below should be read as “row-oriented vs. column-oriented access pattern,” **not** as “PostgreSQL vs. ClickHouse.” A real deployment of either product could perform differently due to engine-specific optimizations (query planners, compression, parallelism, disk I/O) not captured by these substitutes.

Motivation

The SAW architecture designs its memory layer around PostgreSQL with pgvector, but this has never been benchmarked for the kind of query an approval-gate audit trail actually needs: “give me all approval events for agent X within time window Y.” External training on ClickHouse (Aug 18, 2026) suggested a column-oriented backend might serve this query pattern faster at scale. This experiment tests that suggestion directly instead of leaving it as an untested assumption.

Method

- **Synthetic data:** 500,000 events, each with `event_id`, `agent_id` (1–7, matching SAW’s seven-agent orchestration pipeline), `event_type` (approval_request / approval_granted / approval_denied / tool_call), and a timestamp spread across a 30-day synthetic window.
- **Random seed:** 42, fixed for full reproducibility of both data generation and trial queries.
- **Query tested:** `COUNT(*)` of events for a specific agent within a specific time window — a realistic audit-trail query (e.g., “how many approval events did agent 3 generate in the last 3 days”).
- **Trials:** 50 randomized (agent, time-window) queries, identical across all three backends, run in immediate succession to minimize environmental noise.
- **Backends compared:**
 - SQLite, no index on (`agent_id`, `ts`)
 - SQLite, with a composite index on (`agent_id`, `ts`)
 - Pandas/NumPy, boolean-mask filtering over in-memory column arrays
- **Correctness check:** every one of the 50 trials was verified to return an identical count across all three backends before any timing was reported. All 50 passed.

Results (milliseconds per query, n = 50 trials, two independent runs)

Backend	Mean (ms)	Median (ms)	Std. dev (ms)
SQLite — unindexed	20.06 – 20.67	20.15– 20.65	1.5–1.8
SQLite — indexed	0.14	0.14– 0.15	0.06
Pandas columnar (in-memory)	1.69– 1.79	1.65– 1.70	0.18– 0.34

(Range reflects two independent runs with the same seed; the underlying data and query set are identical between runs, so this range reflects normal timing variance, not a change in outcome.)

Honest interpretation

- **The column-oriented proxy beat the unindexed row-oriented store by ~12×** (1.7 ms vs. 20 ms) — this supports the general intuition that columnar scanning outperforms an unindexed row scan for this kind of filter query, at this data scale.
- **The indexed row-oriented store beat the column-oriented proxy by ~12×** (0.14 ms vs. 1.7 ms) — and beat the unindexed row store by ~145×

This is a negative result for the original hypothesis. The training that motivated this experiment suggested a column-oriented backend (ClickHouse) as the faster choice for real-time agent-decision queries. At this data scale (500K events) and for this query shape, a properly indexed row-oriented store outperformed the column-oriented approach by a wide margin. The practical implication for SAW: **if the current PostgreSQL/pgvector implementation is missing an index on (agent_id, timestamp) for its audit-trail queries, adding that index is likely a larger and cheaper win than migrating to a columnar**

engine — at least until the event volume grows far beyond what was tested here.

Explicit limitations

- 500,000 events is a moderate scale; column-oriented engines are generally expected to show a larger relative advantage at much higher volumes (tens of millions of rows) or under concurrent write load, neither of which was tested.
- This experiment measures a single query shape (count-by-agent-and-time-window). Other query shapes (e.g., full-text search, joins across agent state) were not tested and may favor different backends.
- SQLite and Pandas are single-node, single-process substitutes; they do not capture network latency, connection pooling, or distributed-query behavior that a production PostgreSQL or ClickHouse deployment would involve.
- As stated above, this is not a benchmark of the named commercial/open-source products — it isolates the row-vs-column architectural question using available local tools.

Reproducibility

Full code (`benchmark.py`) and raw results (`benchmark_results.json`) are included alongside this report. Running `python3 benchmark.py` with the same seed (42) regenerates the same dataset and query set; per-query timings will vary slightly with hardware and system load (as reflected in the two-run range above), but the qualitative result — indexed row store outperforming the columnar proxy — has been confirmed consistent across independent runs.